

Augmented reality representation of virtual user avatars moving in a virtual representation of the real world at their respective real world locations

Christoph Leuze
Nakamir Inc
Palo Alto, CA, USA
cleuze@nakamir.net

Matthias Leuze
Alpinschule Innsbruck
Natters, Austria
matthias.leuze@gmail.com

ABSTRACT

In this work we present an augmented reality (AR) application that allows a user with an AR display to watch another user, flying an airplane in the Microsoft Flight Simulator 2020 (MSFS), at their respective location in the real world. To do that, we take the location data of a virtual 3D airplane model in a virtual representation of the world of a user playing MSFS, and stream it via a server to a mobile device. The mobile device user can then see the same 3D airplane model at exactly that real world location, that corresponds to the location of the virtual 3D airplane model in the virtual representation of the world. The mobile device user can also see the avatar movement updated according to the 3D airplane movement in the virtual world. We implemented the application on both a cellphone and a see-through headset.

CCS CONCEPTS

• **Human-centered computing** → **Mixed / augmented reality**;
Social content sharing; **Collaborative interaction**.

KEYWORDS

Multiuser Interaction, Virtual Environments, Computer Games, Entertainment, Multiverse, Multiplayer, Augmented Reality

ACM Reference Format:

Christoph Leuze and Matthias Leuze. 2021. Augmented reality representation of virtual user avatars moving in a virtual representation of the real world at their respective real world locations. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Emerging Technologies (SIGGRAPH '21 Emerging Technologies)*, August 9–13, 2021, Virtual Event, USA. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3450550.3465337>

1 INTRODUCTION

Virtual representations of the world have been an important tool for planning or artistic applications. Environmental dioramas, miniature models but also geographic maps that were used, e.g., for

strategy games and ornaments, are early forms of virtual representations of the world with characters and objects that could be placed and moved throughout the scene [Begy 2013]. Today, most virtual representations of the world exist on computers as 3D environments in either map applications such as google earth (Google, CA, USA), as gaming applications such as the Microsoft Flight Simulator (MSFS) (Microsoft, WA, USA) or as 3D renderings of the world or subsets of the world in computer games, maps and other software applications. Currently there are already many applications that stream data from the real world to these software applications. E.g. the real world position of a person, car or airplane is visualized as a little symbol on a software map application to aid with path-finding. More recently the real world location of users can be used to allow users to interact with a virtual representation of the world, e.g., in Pokemon Go (Niantic, CA, USA), where the real world location decides, which virtual places the user can currently interact with or what types of creatures the user can see in the virtual world. The other way around, however, has not yet been done or shown in software. A user that only interacts in the virtual world, e.g., by flying an airplane in a flight simulator, driving a car on a virtual 3D rendering of a real world race track or walking around in a 3D rendering of a real world building, can currently not be seen in the real world and no application exists that show a person in the real world which users in the virtual world currently preside at or close to their real world location. In this application we are showing for the first time, how it is possible to connect a virtual representation of the world with the real world and stream data from a user, who moves and interacts only in the virtual world, to the real world so that that an avatar of the user can be seen by people in the real world with the help of a mobile device. Specifically, we are showing an example of a user flying an airplane in MSFS 2020 and making this airplane visible in the real world via an AR application on a mobile device. To do that, we take the location data of a virtual 3D airplane model in a virtual representation of the world of a user playing MSFS, and stream it via a server to a mobile device. The mobile device user can then see an avatar (in our case the same 3D airplane model) at exactly that real world location, that corresponds to the location of the virtual 3D airplane model in the virtual representation of the world. The mobile device user can also see the avatar movement updated according to the 3D airplane movement in the virtual world.

2 METHODS

Our software application consists of three parts (Fig. 1):

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGGRAPH '21 Emerging Technologies, August 9–13, 2021, Virtual Event, USA

© 2021 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-8364-6/21/08...\$15.00

<https://doi.org/10.1145/3450550.3465337>

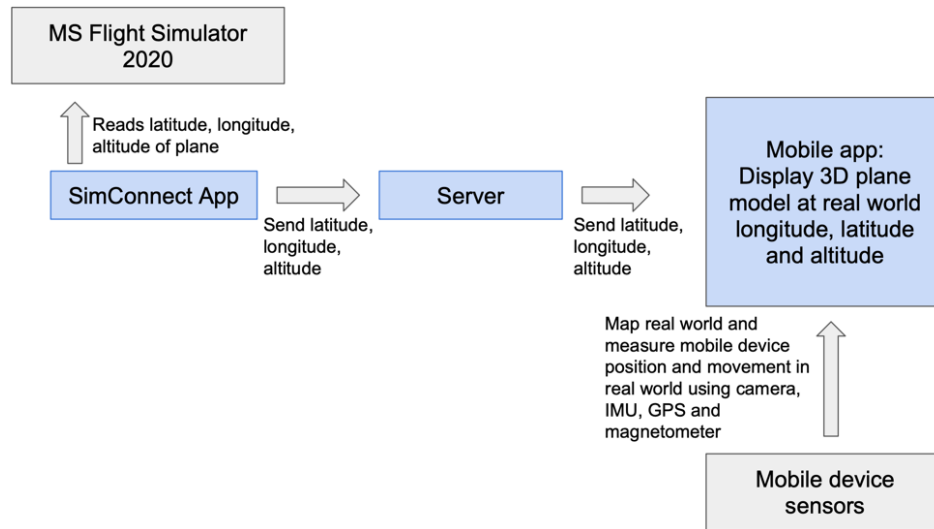


Figure 1: An outline of the application including the three components, the SimConnect app that reads the position information of a virtual airplane in the Microsoft Flight Simulator 2020 at a defined frequency, the Server, which receives this position information and sends it out to a mobile device, and the mobile application that calculates, based on the mobile device sensors (camera, IMU, GPS, magnetometer) and the received position, the avatar (3D airplane model) location and movement in the real world and displays it on the mobile device display.

- an application that reads the position information of a virtual airplane in MSFS and sends it to our server,
- a server that receives the position information and sends it to a mobile device and
- an AR application running on a mobile device that connects to the server, receives the virtual airplane position information, maps out the real-world space to understand its own real-world position and visualizes a 3D model of the virtual airplane in the real world via the mobile device's screen.

2.1 Read information from Flight Simulator

The first application runs on the PC of a user playing MSFS 2020. MSFS 2020 contains a virtual 3D representation of the world and allows the user to fly through this 3D representation of the world with a 3D model of various types of airplanes. The position and altitude of the 3D airplane model is accessible as the latitude, longitude and altitude of the 3D airplane in the virtual world, with latitude and longitude resembling the latitude, longitude and altitude in the real world. Using the FS2020 SDK of MSFS, we developed a separate application that connects to MSFS, extracts at a defined frame-rate the latitude, longitude and altitude information of the virtual airplane and sends it via a UDP protocol to our server. While there were several altitude readings available for the airplane (altitude above ground, altitude above sea level), we chose altitude above sea level as the altitude information of the airplane.

2.2 Server

The server receives the latitude, longitude and altitude information of the virtual airplane from the first application and sends it, via UDP protocol, to any connected clients.

2.3 Mobile AR application on cellphone

The mobile device running the application was an Android phone (Google Pixel 3a, Google, CA, USA), which has sensors including a camera, an inertial measurement unit (IMU), GPS, and a magnetometer. The device measures its real world location using the GPS. The magnetometer acts as a compass and allows the device to understand, in which direction other real world locations and GPS coordinates are located. The camera and IMU are used to map the real world environment including the ground plane around the mobile device and understand its position and movement within this real world environment using simultaneous localization and mapping (SLAM [Smith et al. 1988]) of ARCore (Google, CA, USA). The 3D space, which is built up by the environmental mapping, is anchored to the real world, while movement of the mobile device translates to a movement inside this 3D space. In the mobile AR application we load a 3D avatar, which in our case is the same 3D model of the virtual airplane as used in MSFS. The mobile application then connects to the server and receives the latitude, longitude and altitude information of the virtual airplane flying in MSFS. The mobile application calculates the position of the avatar in the 3D space anchored to the real world based on the GPS coordinates, SLAM position and magnetometer orientation of the mobile device and the GPS coordinates and altitude of the virtual airplane received from the server. To get an accurate altitude of the airplane, we looked up the altitude above sea level of the AR user from the MSFS map and subtracted that AR user altitude from the altitude received from the airplane. We then updated the movement of the avatar in that 3D space based on multiple received real world airplane locations. To allow smooth movement of the avatar at the actual speed of the virtual airplane in MSFS, the mobile AR application always performs a spline interpolation between multiple

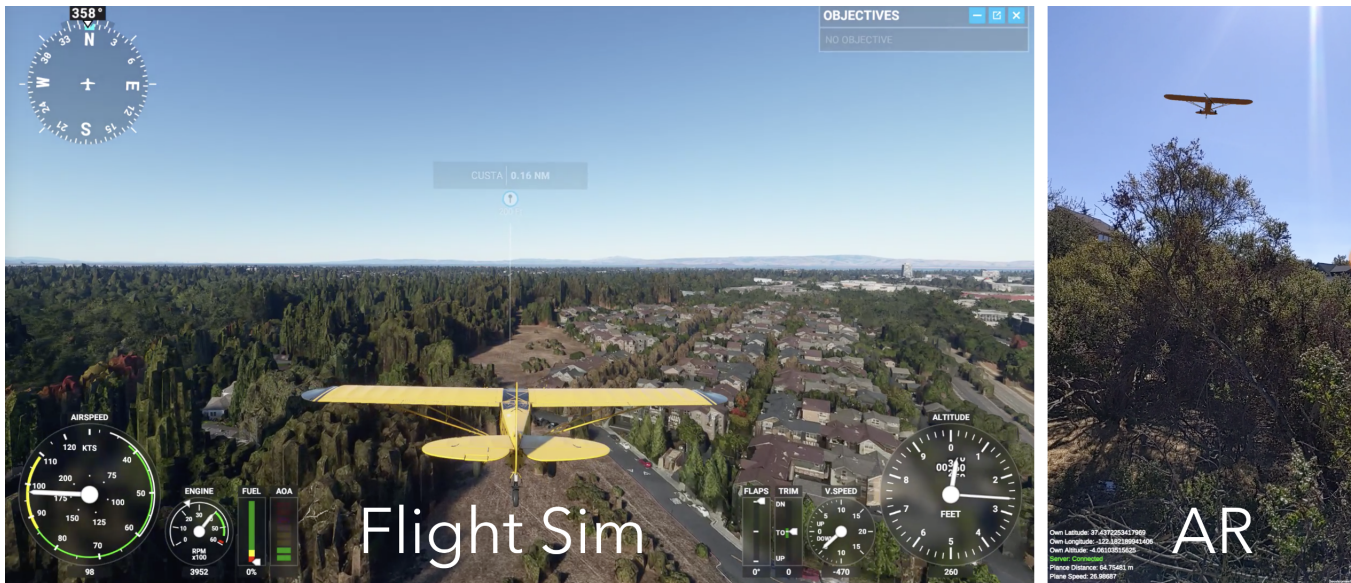


Figure 2: (Left) A view of a 3D model of a virtual airplane flying in Microsoft Flight Simulator 2020. (Right) A view of the app showing the camera stream of the real world and, in the sky, a 3D rendering of the same airplane as in the flight simulator flying in the sky. The location of the 3D model of the airplane in the app is the same world position as the position of the 3D model in the virtual world of the flight simulator. A video of the application running on a smartphone and on a Hololens 2 can be seen on youtube (<https://youtu.be/dDBr96wCmJc>).

received GPS coordinates to interpolate the movements along this path. The speed of the 3D avatar is calculated based on the time stamp when the GPS coordinates were received. The avatar is then moved along this interpolated path at the calculated speed.

2.4 Mobile application on see-through headset

We also tested to run the mobile application on a see-through headset. Because the see-through headset we used (Hololens 2, Microsoft, WA, USA) does not have GPS or a magnetometer, we used the see-through headset in combination with the Google Pixel 3a cellphone used above. The Hololens 2 performed its own SLAM to map the real world surroundings. The Hololens 2 environmental map and the cellphone map were then synchronized by using an image marker to ensure that the origin and coordinate axes of the see-through headset space coincided with the origin and axes of the space mapped by the cellphone app [Sathyanarayana et al. 2020]. Once both spaces were synchronized, the movement path of the avatar in the cellphone app space was streamed via a UDP connection to the see-through headset to update the movement of an identical 3D plane avatar on the see-through display.

3 RESULTS AND DISCUSSION

Figure 2 shows an image of the plane in MSFS on the left and a cellphone screen cap of the AR application with the 3D plane avatar at the corresponding real world location on the right. An example video of the cellphone AR application and the see-through headset application can be seen online (<https://youtu.be/dDBr96wCmJc>). The presented application shows for the first time an example where a virtual model, which exists and moves only in a virtual

representation of the world, can also be seen at the actual location in the real world with the help of an AR application. MSFS 2020 contains a highly realistic virtual representation of the real world, which was constructed from satellite imagery and 3D city data [Lardinois 2020]. A user can freely move within this virtual representation of the real world and real world positioning data like GPS or altitude translates accurately to positions in the virtual world and vice versa. This accurate one-to-one mapping between the virtual and the real world makes MSFS a good candidate to translate also the position of objects that only exist in the virtual world to the real world. However, a similar application does not need to be restricted to MSFS but could either work on a different virtual representation of the real world (e.g. google maps) or even only a subset of the real world, like a city or a single venue. It is only important that there is a well-defined one-to-one mapping between a location in the virtual world and the corresponding location in the real world. Other possible applications could, e.g., be a racing simulation where the virtual race drivers can be seen on the real race track through an AR device or a virtual representation of a real city where the user on the computer exists in this virtual city like, e.g., in a Grand Theft Auto (Rockstar Games, NY, USA) like experience, while the real person can view that virtual user in the real world on a mobile device as long as the virtual user is at a location within viewing distance of the real user. It is also possible that the presented experience goes both ways, not only from the virtual world to the real world, like presented in this work, but also from the real world to the virtual world like for a Pokemon Go like experience. If streaming from both virtual world to real world and real world to virtual world is implemented, this would allow users

in the virtual world and users in the real world to interact and, e.g., perform tasks together that are only possible in one of the worlds.

Our current application still has some limitations: To have realistic movement and calculate the speed of the 3D airplane model, we interpolate a spline curve between multiple received GPS coordinates [Schoenberg 1946]. This leads to a small delay between the position of the 3D virtual airplane in MSFS and the position of the 3D airplane model in the real world (video min 1:49). This delay can be minimized by increasing the frequency, with which the position information is sent and received and by using less points for the spline curve interpolation.

Another limitation is realistic occlusion. While current AR software allows to calculate a depth map of the environment, this usually only works well for close-by objects within less than 10m [Wloka and Anderson 1995]. For this reason, the avatar of the 3D airplane model currently appears always in front of trees and other objects in the environment that would normally occlude the airplane (video min 4:39). This problem could be solved if the mobile application has detailed knowledge of the environment and altitude profiles and, thus, can calculate whether a third object like the airplane is visible to the user [Dünser et al. 2012].

The visibility of the plane in the see-through display was limited when looking at bright objects such as the sky, especially when the airplane was far away (video min 4:11). The video does not depict this limitation accurately and shows a mixed reality capture of the HoloLens 2, which draws the virtual content on top of the camera image. The 3D airplane model visibility in the headset increased in front of darker background such as trees or when it was close by. Unfortunately, display problems in daylight are a common issue for additive light displays and make it difficult to visualize objects that are far away [Le et al. 2019].

To ensure that the AR user does not perceive the airplane to float above or sink underneath the ground, it is important to also provide an accurate altitude of the AR user. However, altitude information provided by the cellphone through GPS is usually much less accurate than the latitude and longitude information [Meade 2000]. MSFS allows to transmit the "altitude above ground", which is unfortunately also no option, because it transmits the altitude to any objects on the ground, e.g. rooftops or trees, leading to the airplane jumping up and down if that altitude value is used. As a solution we looked up the altitude of the AR user in the MSFS world and subtracted that value from the absolute altitude of the airplane, leading to a realistic airplane altitude above ground level.

Latitude and longitude are generally accurate within about 5m of the user [Wing et al. 2005]. While this small discrepancy can usually not be noticed with the airplane up in the sky, it is noticeable when the airplane is taxiing on the ground (video min 5:23). This error is difficult to compensate while relying on GPS and would require additional positioning information such as distances to landmarks and buildings.

In our application we are currently only transmitting GPS and altitude information from the virtual world to the real world user. This allows to depict a realistic movement path of the airplane. For more realistic airplane movement, it would be useful to also transmit information such as the yaw, pitch and roll of the airplane (video min 1:55). While the spline interpolation between the different locations leads to yaw and pitch close to the actual airplane's

yaw and pitch, wind conditions in MSFS and real life can actually lead to the airplane nose pointing in a different direction than the movement direction. When using different airplanes, it would be necessary to send an identifier for the airplane type. For other non-aviation applications, different information might be useful to stream. For indoor environments where GPS might not deliver reliable values, the location information might be more accurately mapped between the virtual and real world based on other sources such as beacons, WiFi information or SLAM. If the 3D avatar is not an airplane, it might be useful to transmit additional information about the avatar, which are completely different depending on the type of avatar. For a humanoid avatar, it might be useful to transmit information like emotions, gaze direction or gestures. Gaming applications might benefit from transmitting player statistics of the user in the virtual world such as the player health or high score statistics. In our example the MSFS user was playing the application on a PC. It would also be possible for the player to play the application in virtual reality. The output streaming to the server and mobile device would not change, whether the player is on a PC or in virtual reality or any other computing device.

4 CONCLUSION

The presented work is an important example of how AR can be used to connect a virtual world, which is a representation of the real world, with the real world. While the presented virtual world was a completely self-contained application running on a PC, it was possible by a user in the real world to view the virtual player on a mobile device as if the virtual player is moving within the real world. This was done by accurately mapping the location information and movement between the virtual world and the real world. Such an application can have important implications not only for gaming applications but also for telepresence, social and collaborative applications where a real world user has an interest in interacting with a spatially separated virtual player.

REFERENCES

- Jason Begy. 2013. Playing at the World: A History of Simulating Wars, People, and Fantastic Adventures: From Chess to Role-Playing Games. *American Journal of Play* (2013).
- Andreas Dünser, Mark Billingham, James Wen, Ville Lehtinen, and Antti Nurminen. 2012. Exploring the use of handheld AR for outdoor navigation. In *Computers and Graphics (Pergamon)*.
- Frederic Lardinois. 2020. Meet the startup that helped Microsoft build the world of Flight Simulator. <https://techcrunch.com/2020/08/17/meet-the-startup-that-helped-microsoft-build-the-world-of-flight-simulator/?guccounter=1>
- Yun-Han Le, Tao Zhan, and Shin-Tson Wu. 2019. Prospects and challenges in augmented reality displays. *Virtual Reality & Intelligent Hardware* 1, 1 (2019), 10–20.
- Mark Meade. 2000. From the Ground Up: Accurate Elevations from GPS. <https://www.pobonline.com/articles/84320-from-the-ground-up-accurate-elevations-from-gps>
- Supriya Sathyanarayana, Christoph Leuze, Brian Hargreaves, Bruce L. Daniel, Gordon Wetzstein, Amit Etkin, Mahendra T Bhati, and Jennifer A McNab. 2020. Comparison of head pose tracking methods for mixed-reality neuronavigation for transcranial magnetic stimulation. *Proceedings of the SPIE medical imaging 2020* (2020), 20.
- Isaac J Schoenberg. 1946. Contributions to the problem of approximation of equidistant data by analytic functions. Part A. On the problem of smoothing or graduation. A first class of analytic approximation formulae. *Quart. Appl. Math.* (1946).
- Randall Smith, Matthew Self, and Peter Cheeseman. 1988. Estimating Uncertain Spatial Relationships in Robotics. In *Machine Intelligence and Pattern Recognition*. 435–461.
- Michael G Wing, Aaron Eklund, and Loren D Kellogg. 2005. Consumer-grade global positioning system (GPS) accuracy and reliability. *Journal of Forestry* 103, 4 (2005), 169–173.
- Matthias M Wloka and Brian G Anderson. 1995. Resolving occlusion in augmented reality. In *Proceedings of the Symposium on Interactive 3D Graphics*. 5–12.